



COMPLETE INTERVIEW GUIDE

Top 50

Generative AI

Interview Questions & Answers

Master Generative AI concepts with comprehensive
Q&A

Fundamentals

LLMs

Prompt Engineering

Fine-tuning

RAG

Production



Naved Khan
Senior Gen-AI Engineer

+ Follow

SECTION 1

Generative AI Fundamentals

Core concepts including GANs, VAEs, Transformers, diffusion models, and evaluation metrics. Understanding the building blocks of generative AI.

Questions Q1-Q10



Naved Khan
Senior Gen-AI Engineer

+ Follow

2/58

What is Generative AI and how does it differ from discriminative models?

- ▶ Generative AI creates brand new content by learning patterns from data - like writing stories or drawing pictures
- ▶ Discriminative models learn to tell the difference between categories - they can classify things but can't create new content
- ▶ Generative models learn how data is structured - they can both classify and create new things
- ▶ Main difference: Discriminative models focus on getting classification right; Generative models focus on learning how data looks
- ▶ When to use: Discriminative for sorting things into categories; Generative for creating new content or adding more training data



Naved Khan
Senior Gen-AI Engineer

+ Follow

3/58

Explain the architecture and working principle of Generative Adversarial Networks (GANs)

- ▶ Two networks compete: The Generator creates fake data, while the Discriminator tries to tell real from fake
- ▶ Training works like a competition: The generator tries to create fake data that looks real, while the discriminator tries to catch the fakes
- ▶ The goal is for the generator to get so good that the discriminator can't tell the difference between real and fake data
- ▶ Common problems: Generator might only create similar things, training can be unstable, or they might not reach a good balance
- ▶ Better versions: WGAN and Progressive GANs fix these problems and make training more stable



Naved Khan
Senior Gen-AI Engineer

+ Follow

4/58

What are Variational Autoencoders (VAEs) and how do they differ from GANs?

- ▶ VAEs compress data into a smaller space and then expand it back - this lets them recreate and generate new data
- ▶ How it works: The encoder squeezes data into a compressed form, the decoder expands it back to full size
- ▶ Training uses two goals: Make sure the output looks like the input, and keep the compressed space organized
- ▶ Main differences: VAEs train more reliably and smoothly; GANs create sharper, more realistic images
- ▶ Best uses: VAEs are great for organizing data in a structured way; GANs are great for creating high-quality images



Naved Khan
Senior Gen-AI Engineer

+ Follow

5/58

What is the role of Transformers in Generative AI?

- ▶ Self-attention lets the model process everything at once and understand connections between words that are far apart
- ▶ Three main types: Decoder-only like GPT (generates text), Encoder-decoder like T5 (translates/summarizes), Encoder-only like BERT (understands text)
- ▶ Big advantages: Can process everything together (unlike older models), understands long text better, can grow bigger
- ▶ Foundation for LLMs: All modern language models like GPT and BERT are built on Transformers
- ▶ Made big models possible: Led to GPT-3, GPT-4, Claude, and other powerful AI models we use today



Naved Khan
Senior Gen-AI Engineer

+ Follow

6/58

What is the latent space in generative models and why is it important?

- ▶ Latent space is a compressed version of data that keeps the important features in a smaller form
- ▶ Key features: Makes data smaller, organizes similar things together, allows smooth changes
- ▶ Controlled generation: By changing values in the latent space, you can control what gets created
- ▶ Interpolation: Moving smoothly between two points in latent space creates smooth transitions in the output
- ▶ Useful for: Finding unusual data, creating new samples efficiently, and making semantic changes



Naved Khan
Senior Gen-AI Engineer

+ Follow

7/58

What are the main applications of Generative AI in real-world scenarios?

- ▶ **Content creation:** Writing articles and code, creating images with DALL-E, making music and videos
- ▶ **Data augmentation:** Creating fake but realistic data when you don't have enough real data to train models
- ▶ **Scientific research:** Helping discover new drugs, designing materials, and creating protein structures
- ▶ **Software development:** Tools like GitHub Copilot that write code, complete code, and help fix bugs
- ▶ **Other uses:** Personalizing content, teaching students, and creating game content



Naved Khan
Senior Gen-AI Engineer

+ Follow

8/58

What is mode collapse in GANs and how can it be prevented?

- ▶ Mode collapse happens when the generator only creates a few similar things instead of variety
- ▶ Signs to watch for: All generated images look the same, same patterns repeating, not covering different types
- ▶ Why it happens: The discriminator becomes too strong, training gets unbalanced, or the loss function has problems
- ▶ How to prevent it: Use better GAN versions like Unrolled GANs, WGAN, or progressive training methods
- ▶ Best practices: Keep training balanced, add regularization, and watch how diverse the outputs are



Naved Khan
Senior Gen-AI Engineer

+ Follow

9/58

Explain the concept of diffusion models in Generative AI

- ▶ **How it works:** Learn to reverse the process of adding noise - start with random noise and gradually clean it up
- ▶ **Forward diffusion:** Slowly adds noise to an image step by step until it's completely random
- ▶ **Reverse diffusion:** The model learns to predict what noise was added and remove it step by step
- ▶ **Big advantages:** Training is stable and reliable, creates very high quality images (often better than GANs), very flexible
- ▶ **Popular versions:** DDPM, DDIM (faster version), and Latent Diffusion (used in Stable Diffusion)



Naved Khan
Senior Gen-AI Engineer

+ Follow

10/58

What are the key challenges in training large-scale generative models?

- ▶ **Computational challenges:** Need huge amounts of memory, training takes weeks or months, costs millions of dollars
- ▶ **Data challenges:** Need enormous datasets, making sure data is good quality and diverse, dealing with copyright issues
- ▶ **Training problems:** Gradients can disappear or explode, loss function can be very complex and hard to optimize
- ▶ **Evaluation challenges:** Hard to measure quality well, making sure model works on new data, finding and fixing bias
- ▶ **Deployment challenges:** Running the model costs a lot, can be slow, hard to scale to many users



Naved Khan
Senior Gen-AI Engineer

+ Follow

11/58

How do you evaluate the quality of outputs from generative models?

- ▶ For images: Use Inception Score (IS), Fréchet Inception Distance (FID), and CLIP Score to measure quality
- ▶ For text: Use Perplexity, BLEU, ROUGE, METEOR, and BERTScore to measure how good the text is
- ▶ Human evaluation: Ask people to rate quality (Mean Opinion Score) or compare two outputs side by side
- ▶ Challenges: Quality is subjective, depends on context, and there's a tradeoff between variety and quality
- ▶ Best practices: Use several different metrics together, always include human feedback, test on many different inputs



Naved Khan
Senior Gen-AI Engineer

+ Follow

12/58

SECTION 2

Large Language Models (LLMs)

Deep dive into GPT architecture, self-attention, few-shot learning, scaling laws, training processes, and LLM limitations.

Questions Q11-Q20



Naved Khan
Senior Gen-AI Engineer

+ Follow

13/58

What is the architecture of GPT (Generative Pre-trained Transformer) and how does it work?

- ▶ **Architecture:** Uses decoder-only transformer that generates text one word at a time, predicting the next word
- ▶ **Main parts:** Breaks text into tokens (BPE), converts to numbers (embeddings), adds position info, processes through transformer layers
- ▶ **How it generates:** Creates text word by word, only looking at previous words (causal masking prevents cheating)
- ▶ **Training process:** First learns language patterns from huge text, then optionally fine-tunes for specific tasks, can learn from examples
- ▶ **Scaling:** Gets better in predictable ways when you make it bigger, give it more data, or use more computing power



Naved Khan
Senior Gen-AI Engineer

+ Follow

14/58

Explain the self-attention mechanism in Transformers

- ▶ **How it works:** Looks at how similar each word is to others, then combines information based on those similarities
- ▶ **The formula:** Uses queries, keys, and values to figure out which words should pay attention to each other
- ▶ **Multi-head attention:** Uses multiple attention mechanisms at once, each looking for different types of relationships
- ▶ **Big advantages:** Can process everything at the same time, understands connections between far-apart words, easier to understand
- ▶ **Limitation:** Gets slower as text gets longer (quadratic growth), which limits how much text it can handle at once



Naved Khan
Senior Gen-AI Engineer

+ Follow

15/58

What is the difference between decoder-only and encoder-decoder architectures?

- ▶ **Decoder-only:** Only looks at previous words, generates one word at a time, great at learning from just a few examples
- ▶ **Encoder-decoder:** Encoder reads input both ways, decoder generates output word by word, better for translation and Q&A
- ▶ **Main differences:** Decoder-only is simpler and better at learning from examples given in the prompt
- ▶ **Use decoder-only for:** General AI models, text generation, when you want to give examples in the prompt
- ▶ **Use encoder-decoder for:** Translation between languages, summarization, tasks where input and output are clearly different



Naved Khan
Senior Gen-AI Engineer

+ Follow

16/58

How do Large Language Models achieve few-shot and zero-shot learning?

- ▶ **Zero-shot:** Does a task using only instructions - no examples needed, just tell it what to do
- ▶ **Few-shot:** Learns from 1-10 examples you give in the prompt, then applies that pattern to new inputs
- ▶ **How it works:** The model figures out the pattern from examples you show it (in-context learning)
- ▶ **What helps:** Bigger models work better, diverse training data, well-written prompts
- ▶ **Big advantages:** No training needed, very flexible, quick to test ideas, easy for anyone to use



Naved Khan
Senior Gen-AI Engineer

+ Follow

17/58

What are the scaling laws in Large Language Models?

- ▶ **Model size:** Performance improves as models get bigger, but gains get smaller (10x bigger $\approx$ 1.2x better)
- ▶ **Data size:** More training data makes models better, following a predictable pattern
- ▶ **Compute strategy:** Bigger models trained for shorter time often beat smaller models trained longer
- ▶ **Emergent abilities:** New skills like few-shot learning and step-by-step reasoning appear at certain sizes
- ▶ **Chinchilla finding:** Using more data compared to model size makes training more efficient



Naved Khan
Senior Gen-AI Engineer

+ Follow

18/58

How does the training process work for Large Language Models?

- ▶ **Data preparation:** Collect huge amounts of text, clean it up, break it into tokens using BPE or SentencePiece
- ▶ **Pre-training:** Learn to predict the next word in a sentence from massive amounts of text
- ▶ **Training tricks:** Use gradient accumulation, mixed precision, and multiple computers to handle huge models
- ▶ **Monitoring:** Watch loss (how wrong predictions are), perplexity (how surprised the model is), learning rate, and gradients
- ▶ **Fine-tuning:** Optionally train further on specific tasks, instructions, or use RLHF to make outputs safer and better



Naved Khan
Senior Gen-AI Engineer

+ Follow

19/58

What are the different sampling strategies used in LLM text generation?

- ▶ Greedy: Always picks the most likely next word - predictable but can get repetitive
- ▶ Temperature: Controls randomness - lower makes it safer and more predictable, higher makes it more creative
- ▶ Top-k: Randomly picks from the K most likely words, giving some variety
- ▶ Top-p (nucleus): Randomly picks from words that together make up probability p, adapts to context
- ▶ Beam search: Keeps track of multiple possible sequences - slower but produces higher quality results



Naved Khan
Senior Gen-AI Engineer

+ Follow

20/58

What is the context window in LLMs and how does it affect model capabilities?

- ▶ **What it is:** The maximum amount of text the model can look at in one go
- ▶ **Limitations:** Has a fixed size, memory use grows very fast with length, hard to train on long texts
- ▶ **Impact:** Limits how long documents it can process, how much conversation history it remembers, how many examples you can give
- ▶ **Workarounds:** Break text into chunks, use sliding windows, process hierarchically, or use RAG to add external knowledge
- ▶ **Recent improvements:** GPT-4 Turbo handles 128K tokens, Claude 3.5 Sonnet handles 200K tokens



Naved Khan
Senior Gen-AI Engineer

+ Follow

21/58

How do LLMs handle different languages and multilingual capabilities?

- ▶ **Training:** Learn from text in many languages, share knowledge through subword tokenization that works across languages
- ▶ **How it works:** Knowledge learned in one language helps with others, similar meanings map to similar places in the model
- ▶ **What it can do:** Translate between languages, understand text in multiple languages, handle mixing languages
- ▶ **Challenges:** Some languages have more training data than others, cultural differences, languages with little data
- ▶ **How to improve:** Balance training data across languages, create specialized models, fine-tune for specific languages, test properly



Naved Khan
Senior Gen-AI Engineer

+ Follow

22/58

What are the main limitations and challenges of Large Language Models?

- ▶ **Hallucinations:** Make up information that sounds believable but is actually wrong
- ▶ **Context limitations:** Fixed size limits mean it can't process very long documents or remember long conversations
- ▶ **Bias:** Copy and make worse the biases found in the training data
- ▶ **Other problems:** Doesn't know recent events, can give different answers to same question, security risks
- ▶ **How to fix:** Use RAG for up-to-date info, fact-check outputs, detect bias, validate inputs, monitor usage



Naved Khan
Senior Gen-AI Engineer

+ Follow

23/58

SECTION 3

Prompt Engineering

Techniques for designing effective prompts, chain-of-thought, few-shot learning, security, and advanced prompting methods.

Questions Q21-Q28



Naved Khan
Senior Gen-AI Engineer

+ Follow ^{24/58}

What is prompt engineering and why is it important?

- ▶ **What it is:** Writing good instructions to get the outputs you want without retraining the model
- ▶ **Why it matters:** Saves money, anyone can do it, quick to try different things, gives you control
- ▶ **Key principles:** Be clear and specific, give context, show examples, organize well, keep improving
- ▶ **Techniques:** Zero-shot (just instructions), few-shot (with examples), chain-of-thought (show reasoning), role-playing, format control
- ▶ **Impact:** Can work as well as or better than fine-tuned models when done right



Naved Khan
Senior Gen-AI Engineer

+ Follow

25/58

What are the best practices for designing effective prompts?

- ▶ **Be clear and specific:** Don't be vague, say exactly what format you want, define what's included
- ▶ **Use examples:** Show 2-5 different examples that are consistent - this teaches the pattern
- ▶ **Provide context:** Give background info, explain constraints, state your goals
- ▶ **Structure logically:** Start with role/context, then task, examples, input, and what you want in output
- ▶ **Keep improving:** Try different versions, check the results, make small improvements based on what works



Naved Khan
Senior Gen-AI Engineer

+ Follow

26/58

Explain Chain-of-Thought prompting and its benefits

- ▶ **How it works:** Ask the model to show its thinking steps before giving the final answer
- ▶ **Benefits:** More accurate on hard problems, you can see how it thinks, easier to spot mistakes
- ▶ **Techniques:** Zero-shot CoT (just ask for steps), few-shot CoT (show examples with steps), auto-CoT, self-consistency
- ▶ **Best for:** Math problems, logic puzzles, writing code
- ▶ **Limitations:** Doesn't always help, can make mistakes in reasoning, makes outputs longer



Naved Khan
Senior Gen-AI Engineer

+ Follow

27/58

What is few-shot learning and how does it work in prompting?

- ▶ **What it is:** Put 1-10 examples in your prompt to show the model what you want
- ▶ **How it works:** The model sees the pattern in your examples and applies it to new inputs
- ▶ **What matters:** Good examples that are different from each other, the order you put them, how many, how hard the task is, model size
- ▶ **Advantages:** No training needed, very flexible, quick to test, easy for anyone to use
- ▶ **Limitations:** Limited by context size, can be inconsistent, not really learning, picking good examples is crucial



Naved Khan
Senior Gen-AI Engineer

+ Follow

28/58

How do you handle prompt injection attacks and security concerns?

- ▶ **Types of attacks:** Direct injection (user tries to override), indirect injection (hidden in data), jailbreaking (tricking safety)
- ▶ **Impact:** Can leak private data, make model do wrong things, bypass safety rules
- ▶ **How to prevent:** Check all inputs, separate user input from system prompts, filter outputs, design prompts carefully
- ▶ **System design:** Give minimal access needed, isolate components, limit request rate, watch for problems
- ▶ **Best practices:** Use multiple layers of protection, monitor constantly, test regularly, teach users about risks



Naved Khan
Senior Gen-AI Engineer

+ Follow

29/58

What are the differences between zero-shot, few-shot, and fine-tuning approaches?

- ▶ **Zero-shot:** Just give instructions, no examples - fastest but not as accurate
- ▶ **Few-shot:** Give 1-10 examples - better accuracy, flexible, no training needed
- ▶ **Fine-tuning:** Actually updates the model - best accuracy, very consistent, but needs training data
- ▶ **Performance ranking:** Fine-tuning is best, then few-shot, then zero-shot
- ▶ **How to choose:** Based on how accurate you need to be, if you have training data, and how flexible you need to be



Naved Khan
Senior Gen-AI Engineer

+ Follow

30/58

How do you optimize prompts for specific tasks like summarization, question answering, and code generation?

- ▶ For summarization: Say how long you want it, what to focus on, what format; include the original text
- ▶ For question answering: Give context clearly, tell it to say "I don't know" when unsure, ask for sources
- ▶ For code generation: Specify programming language, what it should do, coding style; ask for comments and tests
- ▶ How to optimize: Keep improving prompts, test different versions, measure results, adapt to your domain
- ▶ Best practices: Understand what you need, test a lot, write down prompts that work well



Naved Khan
Senior Gen-AI Engineer

+ Follow

31/58

What are advanced prompting techniques like Tree of Thoughts and Self-Consistency?

- ▶ **Tree of Thoughts:** Tries multiple ways of thinking about a problem, evaluates each, picks the best one
- ▶ **Self-Consistency:** Generates several chain-of-thought solutions, picks the answer that appears most often
- ▶ **Other techniques:** ReAct (combines thinking and actions), program-aided (uses code), verification (checks answers)
- ▶ **Advantages:** Works better on hard problems, more reliable results
- ▶ **Limitations:** Costs more to run, takes longer, harder to set up



Naved Khan
Senior Gen-AI Engineer

+ Follow

32/58

SECTION 4

Fine-tuning and Training

Fine-tuning strategies, LoRA, transfer learning, overfitting prevention, hyperparameter tuning, and data preparation.

Questions Q29-Q36



Naved Khan
Senior Gen-AI Engineer

+ Follow

33/58

What is fine-tuning and when should you use it?

- ▶ **What it is:** Updates a pre-trained model by training it more on your specific task data
- ▶ **When to use:** For production apps, when you have lots of data, for specialized tasks, when you need consistent results
- ▶ **How it works:** Start with a pre-trained model, train it on your task data, update the model weights
- ▶ **Advantages:** Best performance, very consistent outputs, learns complex patterns for your task
- ▶ **Disadvantages:** Needs labeled training data, costs money, less flexible, can memorize training data



Naved Khan
Senior Gen-AI Engineer

+ Follow

34/58

Explain LoRA (Low-Rank Adaptation) and its benefits

- ▶ **How it works:** Keeps original model weights frozen, adds small trainable matrices (A and B) that adapt the model
- ▶ **The update:** Only trains the small A and B matrices, original weights stay the same
- ▶ **Benefits:** Uses 100-1000x fewer parameters, trains much faster, needs less memory
- ▶ **Advantages:** Can have different adapters for different tasks, keeps original model abilities
- ▶ **Best for:** Fine-tuning huge models when you don't have lots of computing power or memory



Naved Khan
Senior Gen-AI Engineer

+ Follow

35/58

What is transfer learning in the context of LLMs?

- ▶ **How it works:** First train on huge amounts of general text, then fine-tune on your specific task
- ▶ **Advantages:** Need less data for your task, trains faster, works better than training from scratch
- ▶ **Types:** Feature extraction (use model as feature extractor), fine-tuning (update weights), prompt-based (just use prompts)
- ▶ **How it transfers:** Takes knowledge learned about language and reasoning and applies it to your new task
- ▶ **Why it matters:** Essential for modern LLM apps, makes it possible to adapt models to new tasks effectively



Naved Khan
Senior Gen-AI Engineer

+ Follow

36/58

How do you prevent overfitting when fine-tuning LLMs?

- ▶ The problem: Models memorize training examples instead of learning patterns that work on new data
- ▶ Techniques: Stop training early, adjust learning rate over time, use dropout, add weight decay
- ▶ Other methods: Augment your data, use smaller learning rates, freeze early layers of the model
- ▶ How to watch for it: Compare training performance to validation performance - big gap means overfitting
- ▶ Balance: Some overfitting is normal, but too much means it won't work well on new data



Naved Khan
Senior Gen-AI Engineer

+ Follow

37/58

What are the key hyperparameters to tune during fine-tuning?

- ▶ **Learning rate:** Most important - usually very small ($1e-5$ to $1e-3$), much smaller than pre-training
- ▶ **Batch size:** How many examples at once - affects how stable training is, usually 8-32 for big models
- ▶ **Epochs:** How many times to go through training data - watch validation to know when to stop
- ▶ **Weight decay:** How much to regularize - prevents overfitting, usually 0.01-0.1
- ▶ **Other settings:** Warmup steps (start slow), gradient accumulation (simulate larger batches) - start with recommended values



Naved Khan
Senior Gen-AI Engineer

+ Follow

38/58

What is instruction tuning and how does it differ from task-specific fine-tuning?

- ▶ **What it is:** Trains models to understand and follow instructions, responding in the format you want
- ▶ **Difference:** Task-specific fine-tuning focuses on one task; instruction tuning uses many different examples
- ▶ **How it works:** Collect many different instruction-output pairs, train model to follow instructions
- ▶ **Benefits:** Gets better at zero-shot and few-shot learning, handles new instructions better, more versatile
- ▶ **Examples:** 'Summarize this', 'Translate to French', 'Write Python code' - all different types of instructions



Naved Khan
Senior Gen-AI Engineer

+ Follow

39/58

Explain the concept of catastrophic forgetting in fine-tuning

- ▶ The problem: Fine-tuning makes the model forget what it learned before
- ▶ Why it happens: Updating weights that were good for general knowledge overwrites those patterns
- ▶ What you see: Model does great on your fine-tuning task but worse on general tasks
- ▶ How to prevent: Use lower learning rates, freeze early layers, fine-tune on multiple tasks, use LoRA
- ▶ The balance: Trade-off between doing well on your task and keeping general abilities



Naved Khan
Senior Gen-AI Engineer

+ Follow

40/58

What are the best practices for preparing data for fine-tuning?

- ▶ **Collection:** Get diverse examples that represent your task, need enough (1000+), include edge cases
- ▶ **Cleaning:** Remove duplicates, fix formatting problems, handle missing data, check quality
- ▶ **Formatting:** Use consistent structure, make input-output pairs clear, tokenize properly
- ▶ **Splitting:** Use 70-80% for training, 10-15% for validation, 10-15% for testing
- ▶ **Other tips:** Balance your data, augment if needed, label accurately - data quality is crucial for success



Naved Khan
Senior Gen-AI Engineer

+ Follow

41/58

SECTION 5

RAG – Retrieval Augmented Generation

Retrieval augmented generation pipeline, vector databases, embeddings, hybrid search, and RAG evaluation.

Questions Q37-Q43



Naved Khan
Senior Gen-AI Engineer

+ Follow

42/58

What is RAG and why is it important?

- ▶ **What it is:** Finds relevant information from external sources and adds it to the prompt
- ▶ **How it works:** User asks question → System finds relevant documents → Adds info to prompt → Model generates answer
- ▶ **Why it matters:** Reduces made-up information, provides up-to-date info, specializes for your domain
- ▶ **Benefits:** More accurate answers, current information, responses you can trust
- ▶ **Impact:** Essential for production apps that need to be factually correct



Naved Khan
Senior Gen-AI Engineer

+ Follow

43/58

How does the RAG pipeline work?

- ▶ **Retrieval:** Convert documents and queries to embeddings (numbers), find similar documents using vector search
- ▶ **Augmentation:** Format the found documents as context, add them to the prompt with the query
- ▶ **Generation:** Send the prompt with context to the LLM, generate answer using the retrieved information
- ▶ **Main parts:** Embedding model (converts text to numbers), vector database (stores and searches), LLM (generates answer)
- ▶ **How to improve:** Rewrite queries, re-rank results, use hybrid search, do multi-step retrieval



Naved Khan
Senior Gen-AI Engineer

+ Follow

44/58

What vector databases are commonly used in RAG?

- ▶ **Pinecone:** Fully managed cloud service - best for production when you don't want to manage infrastructure
- ▶ **Chroma:** Lightweight, can embed in your app, open-source - best for development and small projects
- ▶ **Weaviate:** Open-source with cloud options, can create vectors automatically, uses GraphQL API
- ▶ **Qdrant:** High-performance open-source (built in Rust) - great at filtering and searching
- ▶ **Milvus:** Enterprise-grade, can handle billions of vectors - best for very large scale systems



Naved Khan
Senior Gen-AI Engineer

+ Follow

45/58

How do you choose embedding models for RAG?

- ▶ OpenAI: text-embedding-ada-002 (1536 dimensions), text-embedding-3-small/large - good quality, easy to use
- ▶ BGE models: Best open-source models, come in different sizes, can run on your own servers
- ▶ Sentence-Transformers: all-MiniLM-L6-v2 (very fast), all-mpnet-base-v2 (better quality) - popular open-source
- ▶ What to consider: Quality of embeddings, size of vectors, if it fits your domain, speed, cost, language support
- ▶ Best practice: Start with a good general model, try domain-specific models if needed



Naved Khan
Senior Gen-AI Engineer

+ Follow

46/58

What is hybrid search in RAG?

- ▶ **Dense retrieval:** Uses embeddings to find similar meaning - understands synonyms and different ways of saying things
- ▶ **Sparse retrieval:** Uses keyword matching (BM25) - great at finding exact terms and specific identifiers
- ▶ **Hybrid:** Combines both methods - mixes semantic similarity scores with keyword matching scores
- ▶ **Reciprocal Rank Fusion:** Another way to combine results using the ranking positions from each method
- ▶ **Benefits:** Works better than either method alone, handles many different types of queries well



Naved Khan
Senior Gen-AI Engineer

+ Follow

47/58

How do you evaluate RAG system performance?

- ▶ **Retrieval metrics:** Precision (how many retrieved are relevant), recall (how many relevant were found), MRR (ranking quality)
- ▶ **Generation metrics:** Faithfulness (does answer match context), answer relevance (does it answer question), context relevance (is context useful)
- ▶ **RAGAS framework:** Automated way to evaluate RAG systems using LLMs to check multiple metrics
- ▶ **End-to-end:** Check if answers are correct, if users are happy, if tasks get completed successfully
- ▶ **How to evaluate:** Test with many different queries, use automated metrics and also get human feedback



Naved Khan
Senior Gen-AI Engineer

+ Follow

48/58

What are common challenges in RAG systems and how to address them?

- ▶ **Chunking problems:** Important info might be split across chunks - use overlapping chunks, chunk by meaning
- ▶ **Retrieval failures:** Relevant documents not found - use hybrid search, expand queries with synonyms
- ▶ **Context limits:** Can only fit so much in the prompt - compress information, re-rank to get best chunks
- ▶ **Lost in middle:** LLMs pay less attention to middle of context - put most important info at beginning or end
- ▶ **Other challenges:** Model still makes things up even with context, information gets outdated, needs multi-step reasoning



Naved Khan
Senior Gen-AI Engineer

+ Follow

49/58

SECTION 6

Evaluation and Production

Metrics, production deployment, cost considerations, bias handling, RLHF, monitoring, and ethical considerations.

Questions Q44-Q50



Naved Khan
Senior Gen-AI Engineer

+ Follow

50/58

What metrics are used to evaluate generative AI models?

- ▶ For text: Perplexity (how surprised model is), BLEU, ROUGE, METEOR, BERTScore (compare to reference)
- ▶ For images: FID (lower is better - measures quality), IS (higher is better - measures variety), CLIP Score
- ▶ Human evaluation: Ask people to rate quality (Mean Opinion Score), compare outputs side by side
- ▶ Task-specific: For code check if it compiles, for dialogue check if it makes sense and is relevant
- ▶ Best practices: Use several metrics together, always include human feedback, test on many different inputs



Naved Khan
Senior Gen-AI Engineer

+ Follow

51/58

How do you deploy LLMs in production?

- ▶ **Infrastructure:** Host models in cloud or your own servers, create API endpoints, balance load across servers
- ▶ **Optimization:** Use quantization (smaller models), cache results, batch requests, stream responses
- ▶ **Monitoring:** Watch response time, error rates, costs, and quality metrics
- ▶ **Reliability:** Retry failed requests, have backup systems, limit request rates, validate all inputs
- ▶ **Security:** Filter inputs and outputs, require authentication, log everything for auditing



Naved Khan
Senior Gen-AI Engineer

+ Follow

52/58

What are the cost considerations for deploying generative AI?

- ▶ **Cost parts:** Training costs (one-time), inference costs (every time you use the model)
- ▶ **Training costs:** Can cost thousands to hundreds of thousands of dollars depending on model size and data
- ▶ **Inference costs:** Pay per token for APIs, infrastructure costs, costs grow as you scale
- ▶ **How to save:** Cache results, choose right model size, use quantization, batch requests, limit rates
- ▶ **Monitoring:** Track how many tokens used, API calls made, infrastructure costs, set spending limits



Naved Khan
Senior Gen-AI Engineer

+ Follow

53/58

How do you handle bias and fairness in generative AI?

- ▶ Where bias comes from: Training data reflects biases in society, some groups underrepresented, historical biases
- ▶ How to find it: Check outputs for unfair differences, test with diverse inputs, use bias detection benchmarks
- ▶ How to reduce: Use diverse training data, train to be aware of bias, filter biased outputs
- ▶ Other methods: Use careful prompts, have humans review outputs, measure fairness
- ▶ Ongoing process: Keep monitoring and improving - there's no one solution that fixes everything



Naved Khan
Senior Gen-AI Engineer

+ Follow

54/58

What is RLHF (Reinforcement Learning from Human Feedback)?

- ▶ **How it works:** Get human feedback on what's good → Train a model to predict what humans like → Fine-tune with reinforcement learning → Repeat
- ▶ **Main parts:** Collecting human feedback, reward model (predicts what humans like), policy optimization (PPO)
- ▶ **Benefits:** Makes models more helpful, reduces harmful outputs, better at following instructions
- ▶ **Challenges:** Expensive to do, feedback is subjective, can optimize too much, might hurt other abilities
- ▶ **Who uses it:** ChatGPT, Claude 3.5 Sonnet, and other major AI models use RLHF



Naved Khan
Senior Gen-AI Engineer

+ Follow

55/58

How do you monitor and maintain LLM applications in production?

- ▶ **What to watch:** Response time, how many requests per second, error rates, costs, output quality
- ▶ **Alerting:** Set up alerts for problems - slow responses, sudden cost increases, quality getting worse
- ▶ **Logging:** Log everything - inputs and outputs, which model version used, metrics, errors
- ▶ **Maintenance:** Update models, update data (for RAG), improve prompts, fix bugs
- ▶ **Evaluation:** Regularly test performance, do A/B testing, collect user feedback - needs ongoing attention



Naved Khan
Senior Gen-AI Engineer

+ Follow

56/58

What are the ethical considerations in deploying generative AI?

- ▶ **Transparency:** Tell users when AI is being used, explain model limitations, say where data comes from
- ▶ **Privacy:** Protect user data, follow laws like GDPR, collect only what you need
- ▶ **Bias and fairness:** Actively work to reduce bias, make sure everyone treated fairly, watch for unfair differences
- ▶ **Prevent misuse:** Add safeguards to stop harmful uses, monitor for abuse
- ▶ **Other concerns:** Take responsibility for outputs, consider environmental impact, think about job effects



Naved Khan
Senior Gen-AI Engineer

+ Follow

57/58

**Want more
content like this?**



Naved Khan

Senior Gen-AI Engineer

*Tap that follow button and
stay in the loop!*



Like



Comment



Share



Save